

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



BLM4522 PROJE TESLİM RAPORU

Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü
Mehmet Tunahan DÜZYOL
22290109
10.06.2026

Video Bağlantısı: <https://youtu.be/90E0T2PMsdl?si=cMt1plO3ZLZj4eg2>

İçindekiler

1. Giriş	3
1.1 Projenin Amacı	3
1.2 Kapsam	3
1.3 Kullanılan Ortam ve Araçlar	3
2. Erişim Yönetimi ve Kullanıcı Rollerini	4
2.1 SQL Server Authentication ve Windows Authentication Karşılaştırması ..	4
2.2 Login ve User Oluşturma	4
2.3 Rol Tabanlı Yetkilendirme (RBAC)	4
2.4 Yetki Hiyerarşisi ve DENY Uygulaması	4
2.5 Test ve Doğrulama	4
3. Veri Şifreleme (Transparent Data Encryption - TDE)	6
3.1 TDE Nedir ve Nasıl Çalışır	6
3.2 Master Key ve Sertifika Oluşturma	6
3.3 Database Encryption Key (DEK) Oluşturma	6
3.4 TDE Aktivasyonu ve Doğrulanması	6
3.5 Sertifika Yedekleme	6
4. SQL Injection Testleri ve Korunma	8
4.1 SQL Injection Nedir	8
4.2 Güvensiz Prosedür ve Saldırı Simülasyonu	8
4.3 Güvenli Prosedür ile Korunma (Parameterized Query)	8
4.4 Ek Güvenlik Önlemleri (Girdi Doğrulama)	8
5. SQL Server Audit ve Denetim Logları	10
5.1 SQL Server Audit Mimarisi	10
5.2 Server Audit Oluşturma	10
5.3 Database Audit Specification	10
5.4 Server Audit Specification (Login Olayları)	10
5.5 Audit Loglarının Okunması	10
6. Bulgular ve Değerlendirme	12
6.1 Uygulanan Güvenlik Çözümleri Özeti	12
6.2 Sonuç	12
7. Kaynakça	13

1. Giriş

1.1 Projenin Amacı

Bu proje, Microsoft SQL Server (MSSQL) ortamında veritabanı güvenliğini sağlamak için gerekli temel mekanizmaları uygulamayı ve test etmeyi hedeflemektedir. Proje kapsamında kullanıcı erişim kontrolü, hassas verilerin şifrelenmesi (TDE), yaygın güvenlik açıklarından (SQL Injection) korunma ve sunucu/veritabanı hareketlerinin denetlenmesi (Audit) konuları ele alınmıştır.

1.2 Kapsam

Proje dört ana başlık altında yapılandırılmıştır:

- **Erişim Yönetimi (RBAC):** SQL Server ve Windows Authentication modlarının karşılaştırılması, rol tabanlı yetkilendirme ve nesne seviyesinde erişim kısıtlamaları.
- **Veri Şifreleme (TDE):** Veritabanı dosyalarının disk üzerinde şeffaf bir şekilde şifrelenmesi ve anahtar yönetimi hiyerarşisinin kurulması.
- **Uygulama Güvenliği (SQL Injection):** Yaygın bir saldırı türü olan SQL Injection simülasyonu ve parametrelili sorgular ile korunma yöntemlerinin karşılaştırılması.
- **Denetim ve İzleme (SQL Audit):** Sunucu ve veritabanı olaylarını izlemek, loglamak ve denetim kayıtlarını okumak için Audit yapısının oluşturulması.

1.3 Kullanılan Ortam ve Araçlar

Bileşen	Detay
Veritabanı Sistemi	Microsoft SQL Server 2025
Yönetim Arayüzü	SQL Server Management Studio (SSMS)
Örnek Veritabanı	AdventureWorks2025
İşletim Sistemi	Windows 11

2. Eriřim Yönetimi ve Kullanıcı Rollerini

Veritabanı güvenlięinin temelini oluřturan eriřim yönetimi, kullanıcılara "en az yetki prensibi" (Principle of Least Privilege) doęrultusunda gerekli minimum yetkilerin atanmasını saęlar. Bu bölümde, farklı kimlik doęrulama modları kullanılarak kullanıcılar oluřturulmuř ve rol tabanlı yetkilendirme (RBAC) uygulanmıřtır.

2.1 SQL Server Authentication ve Windows Authentication Karřılařtırması

SQL Server, iki temel kimlik doęrulama modunu destekler: SQL Server Authentication ve Windows Authentication. Çalıştırılan SQL komutu ile sunucunun mevcut kimlik doęrulama modu kontrol edilmiř ve sunucunun ya yalnızca Windows Authentication'ı (deęer 1) ya da Karma Modu (Mixed Mode) (deęer 0) kullandıęı tespit edilmiřtir.

- **Windows Authentication (Tümleřik Güvenlik):** Kullanıcıların kimlik doęrulası merkezi bir Windows servisi (Active Directory) üzerinden yapılır. řifreler SQL Server'da saklanmaz, bu da genellikle daha güvenli kabul edilir. Merkezi yönetim ve SSO (Single Sign-On) avantajı sunar.
- **SQL Server Authentication:** Kullanıcı adı ve řifre bilgileri SQL Server içinde saklanır. Windows dıřı ortamlardan veya karma güvenlik gerektiren durumlarda kullanılır. Uygulanan komutlar, bu mod kullanılarak Login'lerin oluřturulmasına odaklanmıřtır.

2.2 Login ve User Oluřturma

SQL Server'da kimlik doęrulama iki ařamalıdır: sunucu seviyesinde Login ve veritabanı seviyesinde User.

1. **Login Oluřturma:** SQL Server Authentication kullanılarak sunucu seviyesinde `raporcu_login`, `yazici_login` ve `yonetici_login` oluřturulmuřtur. Tüm Login'ler için parola politikalarının (`CHECK_EXPIRATION = ON`, `CHECK_POLICY = ON`) etkinleřtirilmesi, parolaların karmařıklıęını ve düzenli deęiřimini zorunlu kılarak temel güvenlik adımıını atmıřtır.
2. **User Oluřturma:** Oluřturulan Login'ler, `AdventureWorks2025` veritabanında `raporcu_user`, `yazici_user` ve `yonetici_user` olarak eřleřtirilmiřtir. Veritabanı içindeki yetkilendirmeler bu User objeleri üzerinden yapılacaktır.

2.3 Rol Tabanlı Yetkilendirme (RBAC)

Oluřturulan veritabanı kullanıcılarına, görev tanımlarına uygun olarak önceden tanımlı veritabanı rolleri atanmıřtır:

- **raporcu_user:** `db_datareader` rolüne atanmıřtır. Bu, kullanıcının veritabanındaki tüm tablolardan yalnızca veri okumasına (SELECT) izin verir.

- **yazici_user:** Hem `db_datareader` hem de `db_datawriter` rollerine atanmıştır. Bu, kullanıcının veri okumanın yanı sıra veri ekleme (INSERT), güncelleme (UPDATE) ve silme (DELETE) işlemleri yapabilmesini sağlar.
- **yonetici_user:** `db_owner` rolüne atanmıştır. Bu, kullanıcıya veritabanı üzerinde tam yetki ve kontrol sağlar.

2.4 Yetki Hiyerarşisi ve DENY Uygulaması

Veritabanı güvenliğinde, genel rollerden (örneğin `db_datareader`) bağımsız olarak belirli nesnelere özel yetki ataması (GRANT) veya açıkça erişimi engelleme (DENY) mümkündür.

- `raporcu_user`'a genel okuma yetkisine ek olarak `Person.Person` tablosu üzerinde ek SELECT yetkisi verilmiştir (`GRANT SELECT ON`).
- DENY Komutunun Kullanımı: Proje kapsamında, hassas veriler içerdiği varsayılan `HumanResources.EmployeePayHistory` tablosuna `raporcu_user` ve `yazici_user` için erişim açıkça engellenmiştir (`DENY SELECT ON`). SQL Server yetki modelinde DENY, herhangi bir GRANT veya rol üyeliğinden gelen yetkiden daha yüksek önceliğe sahiptir (Explicit Deny Overrides Grant). Bu sayede, `yazici_user`'ın `db_datareader` rolünden gelen SELECT yetkisi bile bu özel tablo için geçersiz kılınmıştır.

2.5 Test ve Doğrulama

Yetki yapılandırması, `sys.dm_exec_requests` gibi sistem görünümleri ve `EXECUTE AS USER` komutu ile test edilmiştir.

- **Yetki Sorguları:** `sys.database_permissions` ve `sys.database_role_members` sorguları çalıştırılarak kullanıcıların hangi rollere üye olduğu ve hangi nesneler üzerinde özel izinlere sahip olduğu doğrulanmıştır.
- **EXECUTE AS Testi:** `raporcu_user` bağlamında yapılan testlerde, yetki verilen `Person.Person` tablosuna erişim başarılı olmuştur. Ancak, DENY komutuyla erişimi kısıtlanan `HumanResources.EmployeePayHistory` tablosuna erişim denemesi hata vermiştir. Bu sonuç, DENY kuralının ve rol tabanlı yetkilendirmenin beklenen şekilde çalıştığını kanıtlamıştır.

3. Veri Şifreleme (Transparent Data Encryption - TDE)

Transparent Data Encryption (TDE), veritabanı dosyalarını (veri ve log dosyaları) disk üzerinde şeffaf bir şekilde şifreleyerek, yetkisiz fiziksel erişim durumunda verinin korunmasını sağlar. TDE, uygulamanın kodunda veya sorgularda herhangi bir değişiklik gerektirmez.

3.1 TDE Nedir ve Nasıl Çalışır

TDE, veritabanı motoru seviyesinde çalıştığı için veritabanı yöneticileri ve uygulamalar tarafından görünmezdir (şeffaftır). Veri diskten okunurken şifresi çözülür, diske yazılırken şifrelenir. TDE, bir anahtar hiyerarşisi kullanır:

1. **Service Master Key (SMK):** SQL Server örneği tarafından otomatik oluşturulur ve sunucudaki diğer şifreleme anahtarlarını korur.
2. **Database Master Key (DMK):** Veritabanında oluşturulur ve Sertifikayı şifreler.
3. **Sertifika (Certificate):** DMK tarafından korunur ve her bir veritabanının DEK'ini şifreler.
4. **Database Encryption Key (DEK):** Sertifika tarafından korunur ve veritabanının kendisini şifrelemek için kullanılan simetrik anahtardır.

3.2 Master Key ve Sertifika Oluşturma

Şifreleme zincirinin güvenliğini sağlamak için sırasıyla DMK ve Sertifika oluşturulmuştur:

1. **Service Master Key Kontrolü:** `sys.symmetric_keys` sorgulanarak Service Master Key'in varlığı kontrol edilmiş ve şifreleme zincirinin temeli doğrulanmıştır.
2. **Database Master Key (DMK) Oluşturma:** `master` veritabanında DMK oluşturulmuş ve güçlü bir parola ile şifrelenmiştir. DMK, sunucu düzeyinde bir anahtar deposu görevi görür ve Sertifikaların oluşturulmasında kullanılır.
3. **TDE Sertifikası Oluşturma:** DMK kullanılarak sunucu seviyesinde `TDE_AW2025_Cert` adında bir Sertifika oluşturulmuştur. Bu Sertifika, daha sonra veritabanı şifreleme anahtarını koruyacaktır.

3.3 Database Encryption Key (DEK) Oluşturma

Şifreleme işlemini doğrudan gerçekleştirecek olan simetrik anahtar, hedef veritabanında oluşturulmuştur:

- `AdventureWorks2025` veritabanında, AES_256 algoritması kullanılarak `DATABASE ENCRYPTION KEY` oluşturulmuştur. Bu DEK, bir önceki adımda oluşturulan sunucu sertifikası (`TDE_AW2025_Cert`) ile şifrelenmiştir. AES_256, endüstri standardı güçlü bir şifreleme algoritmasıdır.

3.4 TDE Aktivasyonu ve Doğrulanması

DEK oluşturulduktan sonra TDE özelliği etkinleştirilmiştir:

1. **TDE'nin Etkinleştirilmesi:** `ALTER DATABASE [AdventureWorks2025] SET ENCRYPTION ON;` komutu çalıştırılarak veritabanı dosyalarının şifreleme süreci başlatılmıştır.
2. **Durum Doğrulaması:** `sys.dm_database_encryption_keys` DMV'si sorgulanarak TDE durumu izlenmiştir. Şifreleme tamamlandığında `encryption_state` değerinin 3 (Şifrelenmiş) durumuna geçtiği doğrulanmıştır.

3.5 Sertifika Yedekleme

TDE'nin en kritik güvenlik adımlarından biri, şifreleme Sertifikasının (özel anahtar dahil) yedeklenmesidir:

- Sertifika, `BACKUP CERTIFICATE... WITH PRIVATE KEY` komutu kullanılarak belirtilen dosya yoluna (`C:\Backups\`) güçlü bir parola ile korunarak yedeklenmiştir. Bu adım, sunucunun tamamen bozulması veya veritabanının başka bir sunucuya taşınması durumunda şifreli veriye erişimi sürdürmek için hayati öneme sahiptir. Sertifika kaybedilirse, verinin şifresi çözülemez.

4. SQL Injection Testleri ve Korunma

4.1 SQL Injection Nedir

SQL Injection (SQLi), web veya uygulama girdileri aracılığıyla saldırganın veritabanına yetkisiz SQL komutları göndermesini sağlayan bir güvenlik açığıdır. Bu açık, kullanıcının girdiği verilerin yeterince doğrulanmadan veya sterilize edilmeden doğrudan dinamik bir SQL sorgusuna eklenmesiyle oluşur. SQLi saldırıları veri sızıntısına, veri manipülasyonuna veya servisin durmasına (Denial of Service) yol açabilir.

4.2 Güvensiz Prosedür ve Saldırı Simülasyonu

Proje kapsamında, string birleştirme yoluyla dinamik SQL oluşturan `usp_GetProduct_Unsafe` adında güvensiz bir saklı yordam (Stored Procedure) tanımlanmıştır.

- **Saldırı Simülasyonu 1 (Koşul Atlatma):** Kullanıcı adı parametresi olarak ' OR '1'='1 gönderildiğinde, prosedür içindeki WHERE koşulu `WHERE Name LIKE '%" OR '1'='1%'` haline gelir. `OR '1'='1'` kısmı her zaman doğru olduğu için, sorgu tüm tablo kayıtlarını döndürerek yetkisiz veri erişimine neden olur.
- **Saldırı Simülasyonu 2 (Veri Sızdırma):** Kullanıcı adı parametresi olarak ' UNION SELECT TOP 5 BusinessEntityID, FirstName + ' ' + LastName, 0 FROM Person.Person -- gönderildiğinde, orijinal sorgu sonucuyla birlikte başka bir tablodan (`Person.Person`) veriler sızdırılmıştır. -- işareti, SQL komutunun geri kalanını yorum satırı haline getirerek sintaks hatalarını önlemiştir. Bu simülasyon, SQL Injection'ın veri sızıntısı için nasıl kullanılabileceğini açıkça göstermiştir.

4.3 Güvenli Prosedür ile Korunma (Parameterized Query)

SQL Injection'dan korunmanın en etkili yolu, parametrelili sorgular (Parameterized Queries) kullanmaktır.

- **Güvenli Prosedür:** `usp_GetProduct_Safe` prosedürü, kullanıcı girdisini doğrudan SQL cümlesi içine string olarak eklemek yerine, veritabanı motorunun girdiyi yalnızca bir değer (veri) olarak işlemesini sağlayan parametre bağlama yöntemini kullanmıştır.
- **Doğrulama:** Aynı injection girişimi (' OR '1'='1 ve `UNION SELECT...`) bu güvenli prosedür üzerinde denendiğinde, sistem artık bu girdileri SQL komutu değil, sadece aranacak bir metin dizisi olarak yorumlamıştır. Sonuç olarak, saldırı başarısız olmuş ve beklenen hata mesajları veya boş sonuçlar dönmüştür. Bu, parametrelili sorgu kullanımının güvenlik açısından üstünlüğünü kanıtlamıştır.

4.4 Ek Güvenlik Önlemleri (Girdi Doğrulama)

Güvenlik derinliğini artırmak amacıyla, uygulamadan gelen girdinin veritabanına ulaşmadan önce doğrulanması gerekir.

- `usp_GetProduct_Validated` prosedüründe, ürün adı parametresinin boş olup olmadığı ve fiyat aralığı (Min/Max Price) mantıksal olarak tutarlı olup olmadığı kontrol edilmiştir. Hatalı veya kötü niyetli olabilecek girdiler, `RAISERROR` komutuyla erken aşamada reddedilerek veritabanı yükü azaltılmış ve güvenlik artırılmıştır. Örneğin, Minimum fiyatın maksimum fiyattan büyük olması gibi mantıksal hatalar engellenmiştir.

5. SQL Server Audit ve Denetim Logları

SQL Server Audit, güvenlik olaylarını (başarılı/başarısız login'ler, veri erişimi, şema değişiklikleri) izlemek ve denetim kayıtlarını güvenli bir konumda saklamak için hayati bir araçtır.

5.1 SQL Server Audit Mimarisi

Denetim mimarisi, sunucu üzerindeki olayları yakalamak için üç aşamalı bir yapı kullanır:

1. **Server Audit:** Denetim kayıtlarının hedefi (dosya, Windows Günlüğü) ve genel ayarlarını tanımlar.
2. **Server Audit Specification:** Sunucu seviyesindeki olayları (Login/Logout gibi) izler.
3. **Database Audit Specification:** Veritabanı seviyesindeki olayları (SELECT, INSERT, EXECUTE gibi) izler.

Server Audit, logların nerede saklanacağını (dosya yolu, boyut) belirlerken; Audit Specification'lar ise hangi olayların ve kimin tarafından tetiklenen olayların loglanacağını tanımlar.

5.2 Server Audit Oluşturma

`AW_Proje3_Audit` adında bir `SERVER AUDIT` objesi oluşturulmuştur.

- **Hedef ve Boyut:** Loglar, belirtilen yerel bir dizine (`C:\Backups\AuditLogs\`) yazılmak üzere yapılandırılmış, maksimum dosya boyutu (100 MB) ve otomatik dosya çevrimi (10 dosya) ayarları tanımlanmıştır.
- **Hata Yönetimi:** `ON_FAILURE = CONTINUE` ayarı, denetleme sisteminin herhangi bir nedenle başarısız olması durumunda SQL Server'ın çalışmaya devam etmesini sağlamıştır.
- **Filtreleme:** Audit, DML (INSERT, UPDATE, DELETE), SELECT ve EXECUTE eylemlerini izleyecek şekilde filtrelenmiştir.
- Audit oluşturulduktan sonra `ALTER SERVER AUDIT... WITH (STATE = ON)` komutuyla etkinleştirilmiştir.

5.3 Database Audit Specification

`AdventureWorks2025` veritabanı içindeki özel işlemleri izlemek için `AW_DB_Audit_Spec` oluşturulmuştur.

- **Hassas Veri İzleme:** `HumanResources.EmployeePayHistory` tablosunda `PUBLIC` rolünün `SELECT` işlemleri izlenerek, bu hassas verilere kimin eriştiği kayıt altına alınmıştır.
- **DML İzleme:** `Sales.SalesOrderHeader` tablosu üzerinde `PUBLIC` rolünün tüm DML (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) eylemleri denetime tabi tutulmuştur.
- **Kullanıcı ve Prosedür İzleme:** Belirli bir kullanıcı (`raporcu_user`) tarafından `Person.Person` tablosuna yapılan `SELECT` işlemleri ve güvenli saklı yordam (`usp_GetProduct_Safe`) çalıştırmaları da denetim kapsamına alınmıştır.

5.4 Server Audit Specification (Login Olayları)

Sunucu seviyesindeki bağlantı denemelerini izlemek için `AW_Server_Login_Spec` oluşturulmuştur.

- Bu spesifikasyon, başarılı loginler (`SUCCESSFUL_LOGIN_GROUP`), başarısız loginler (`FAILED_LOGIN_GROUP`) ve çıkışlar (`LOGOUT_GROUP`) olmak üzere üç temel kimlik doğrulama olayını izlemek üzere ayarlanmıştır. Başarısız login denemelerinin loglanması, kaba kuvvet (brute force) saldırı girişimlerinin veya yetkisiz erişim denemelerinin tespiti için kritik öneme sahiptir.

5.5 Audit Loglarının Okunması

Denetim mekanizmalarını test etmek için gerekli sorgular (`SELECT`, `EXECUTE AS USER` bağlamında `SELECT`) çalıştırıldıktan sonra, kayıtlar `sys.fn_get_audit_file` fonksiyonu kullanılarak okunmuştur.

- **Log İçeriği:** Log kayıtları, olayın zamanı, hangi sunucu örneğinde gerçekleştiği, hangi veritabanını etkilediği, işlemi gerçekleştiren kullanıcı (`server_principal_name`), eylemin türü (`action_id`) ve kullanılan SQL ifadesinin bir özeti dahil olmak üzere detaylı bilgiler sunmuştur.
- **Doğrulama:** `raporcu_user` bağlamında yapılan `SELECT` işlemlerinin loglandığı, ayrıca başarısız login denemelerinin `action_id = 'LGF'` filtresiyle kolayca tespit edilebildiği görülmüştür. Bu, oluşturulan denetim yapısının operasyonel güvenliği ve adli analiz yeteneğini (forensic analysis) sağladığını kanıtlamıştır.

6. Bulgular ve Değerlendirme

6.1 Uygulanan Güvenlik Çözümleri Özeti

Alan	Tespit / Uygulama	Sonuç
Erişim Kontrolü	Rol Tabanlı Yetkilendirme (RBAC) ve DENY komutu uygulandı.	<code>raporcu_user</code> 'ın yetkili tabloya erişimi başarılı olurken, DENY uygulanan hassas tabloya erişimi engellenmiştir. En az yetki prensibi sağlanmıştır.
Veri Şifreleme (TDE)	Master Key, Sertifika ve AES_256 Database Encryption Key (DEK) hiyerarşisi kuruldu.	<code>AdventureWorks2025</code> veritabanı, disk üzerinde şeffaf bir şekilde şifrlenmiştir (<code>encryption_state = 3</code>). Kritik Sertifika yedeği alınmıştır.
SQL Injection Korunması	String birleştirmeyeyle oluşturulan güvensiz prosedür saldırıya karşı simüle edildi.	Saldırı girişimleri güvensiz prosedürde veri sızıntısıyla sonuçlanırken, parametrelili sorgu kullanan güvenli prosedürde saldırı engellenmiştir. Parameterized Query'nin önemi kanıtlanmıştır.
Denetim (Audit)	Server Audit, Database Audit Specification ve Server Audit Specification (Login) bileşenleri kuruldu.	Login/Logout, DML ve belirli kullanıcı eylemleri başarıyla loglanmıştır. Başarısız login denemeleri de dahil olmak üzere olayların geriye dönük izlenebilirliği sağlanmıştır.

6.2 Sonuç

Bu proje, Microsoft SQL Server 2025 üzerinde AdventureWorks2025 veritabanı kullanılarak veritabanı güvenliğinin çok katmanlı yaklaşımını başarıyla uygulamıştır. Uygulanan RBAC ile kullanıcılar görevlerine göre yetkilendirilmiş, DENY komutunun üstünlüğü ile hassas verilere erişim kısıtlanmıştır. TDE ile veri ve log dosyaları disk seviyesinde korunmuş, bu sayede fiziksel ortamın ele geçirilmesi durumunda bile veri gizliliği sürdürülmüştür. SQL Injection saldırı simülasyonları ile uygulama katmanı güvenlik açığının tehlikeleri gösterilmiş ve Parameterized Query kullanımının korunma için mutlak gereklilik olduğu vurgulanmıştır. Son olarak, SQL Server Audit mimarisi kurularak sunucu ve veritabanı olaylarının detaylı denetim kaydı alınmış, bu sayede güvenlik ihlali girişimlerinin tespiti ve analizi için zemin hazırlanmıştır.

7. Kaynakça

- Microsoft Docs — CREATE LOGIN ve CREATE USER:
<https://learn.microsoft.com/sql/t-sql/statements/create-login-transact-sql>
- Microsoft Docs — SQL Server Yetkilendirme:
<https://learn.microsoft.com/sql/relational-databases/security/permissions-hierarchy-database-engine>
- Microsoft Docs — Transparent Data Encryption (TDE):
<https://learn.microsoft.com/sql/relational-databases/security/encryption/transparent-data-encryption>
- Microsoft Docs — SQL Injection:
<https://learn.microsoft.com/sql/relational-databases/security/sql-injection>
- Microsoft Docs — SQL Server Audit:
<https://learn.microsoft.com/sql/relational-databases/security/auditing/sql-server-audit-database-engine>